SRE reliability lab

Reliability stated as numbers, then tested by breaking things.

A 5% error incident for 30 minutes pages after 18 minutes and spends 3.47% of the monthly budget.

A latency fault caused zero errors, so an availability-only alert would have stayed silent.

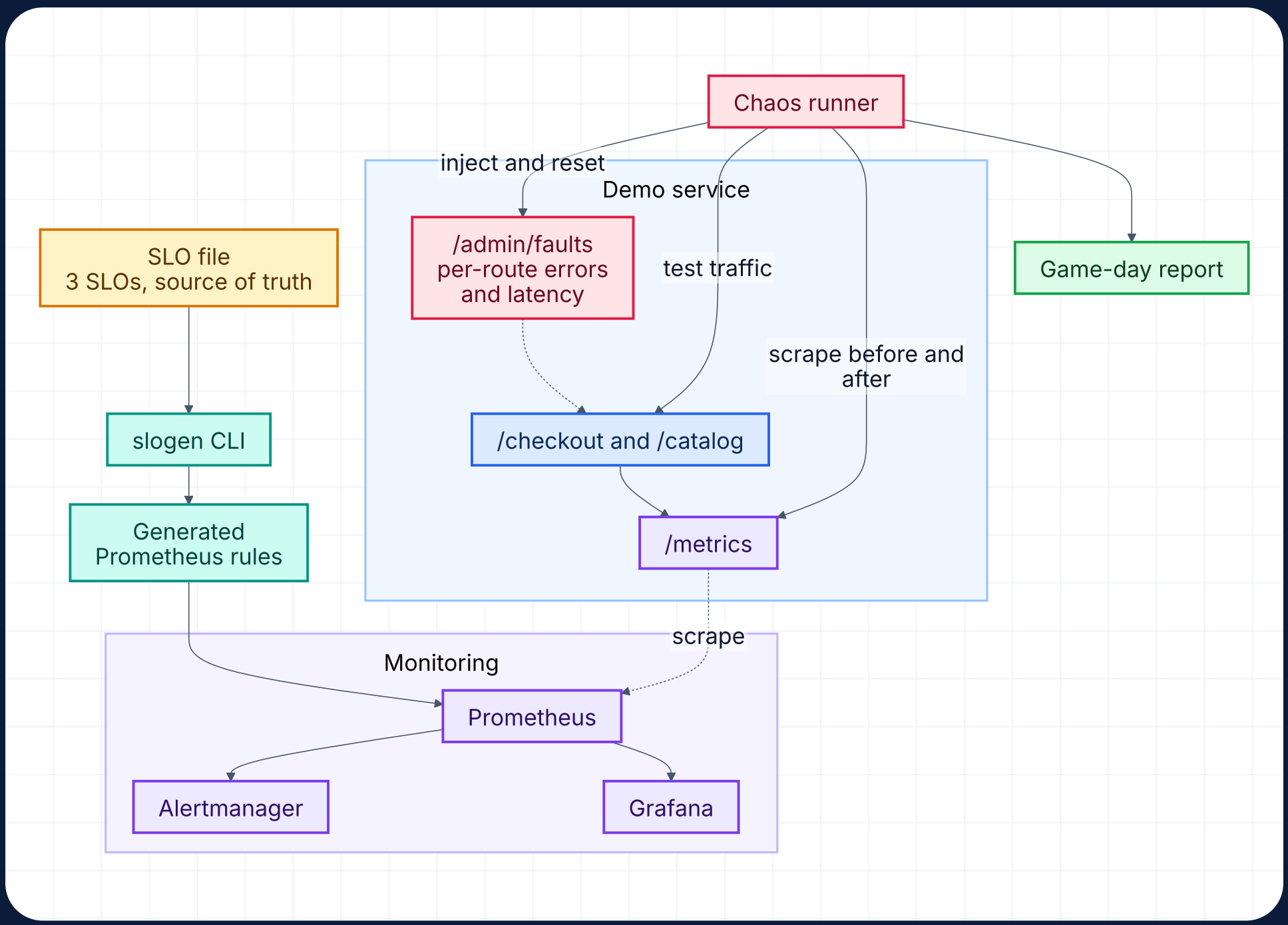
Two game days run, each with a control route and a check against the service's own metrics.

Jaime Castillo Araujo

5 schematics follow. Code: github.com/Joxejaime/projects-2026

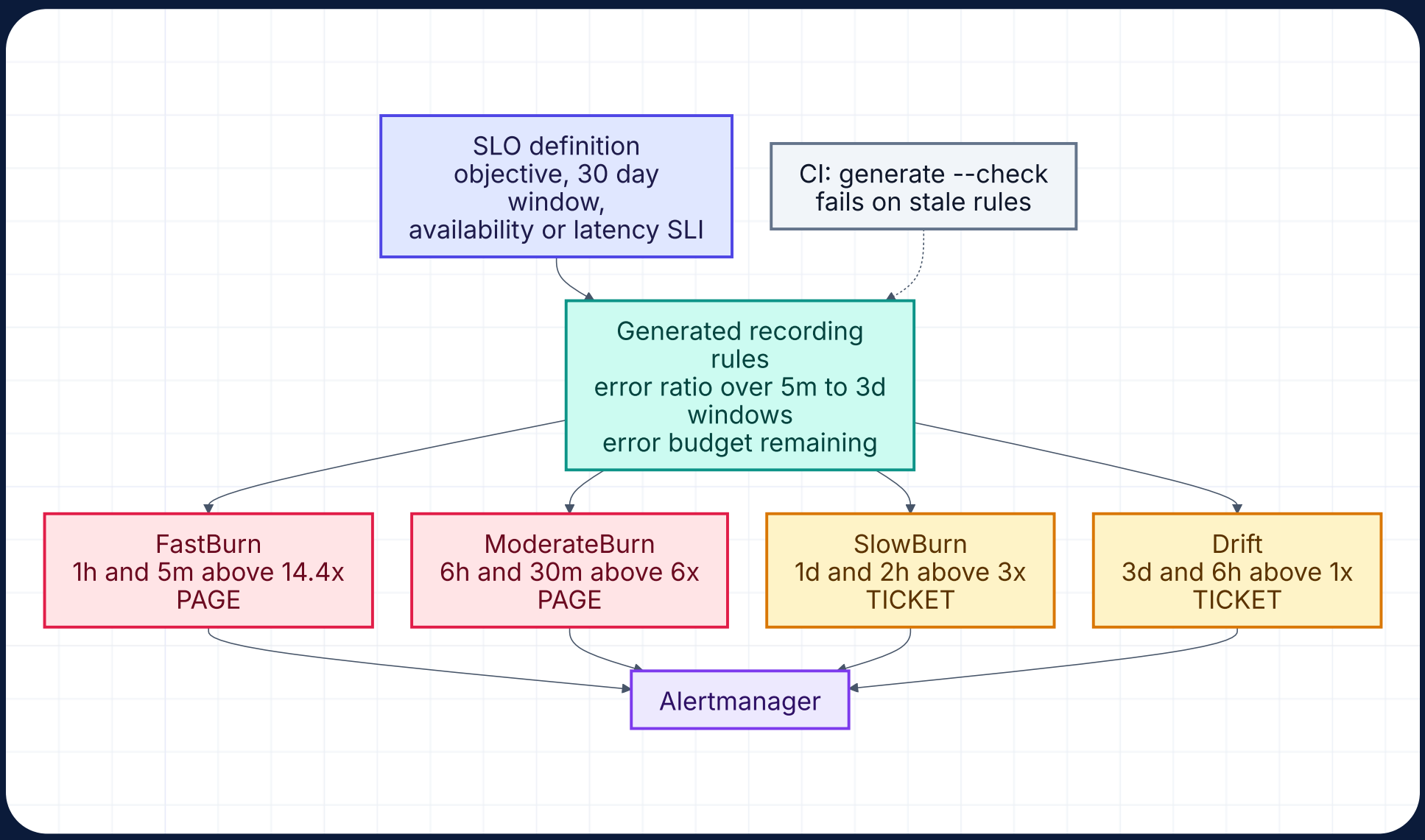
Reliability as code, tested with real failures

One SLO file drives the alerts. A chaos runner breaks the demo service and checks what happened.



One SLO file generates every alert

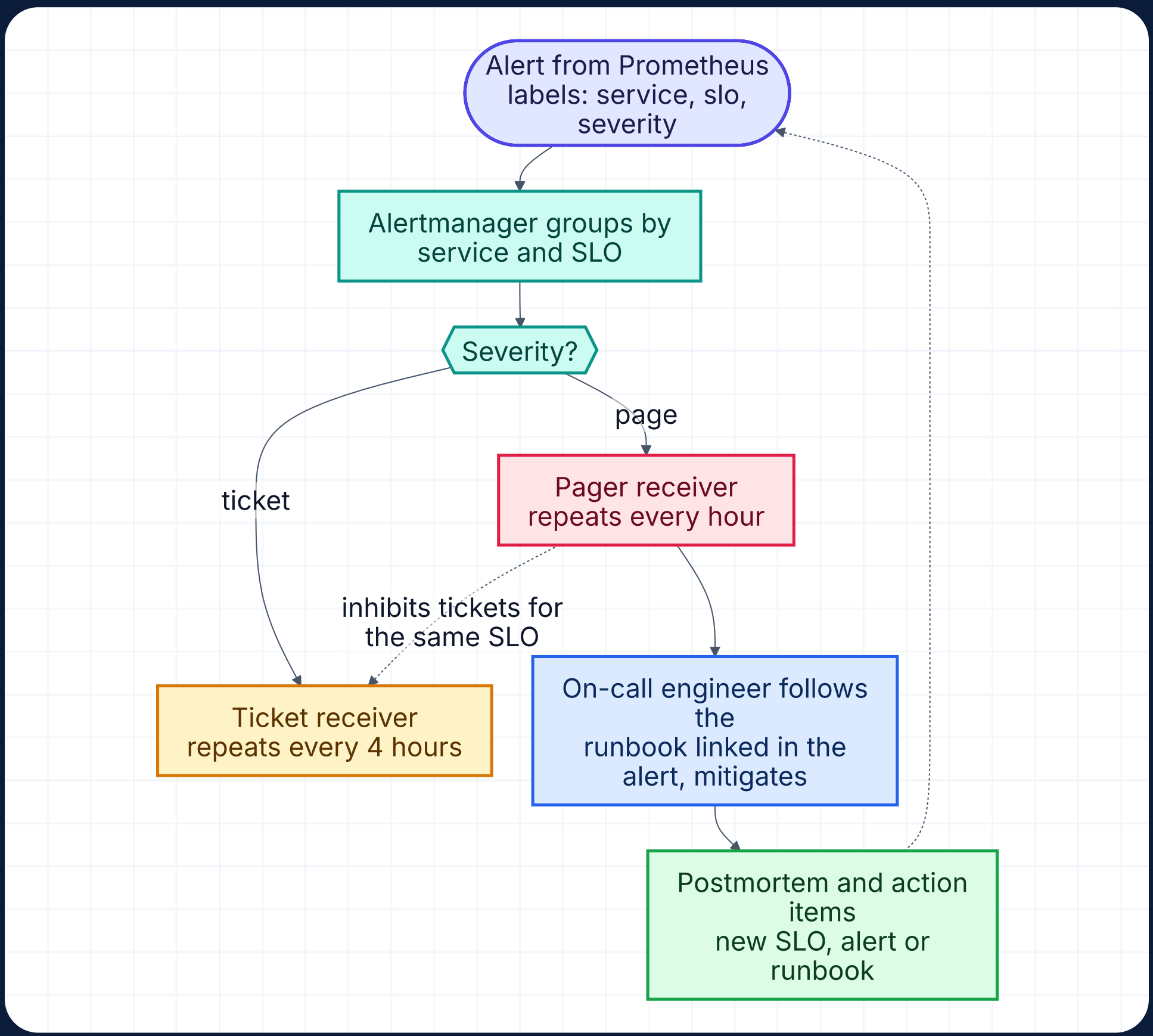
Four burn-rate alerts: two page someone, two open tickets.



SLO definition · Generated rules · Page · Ticket · Alertmanager · CI check

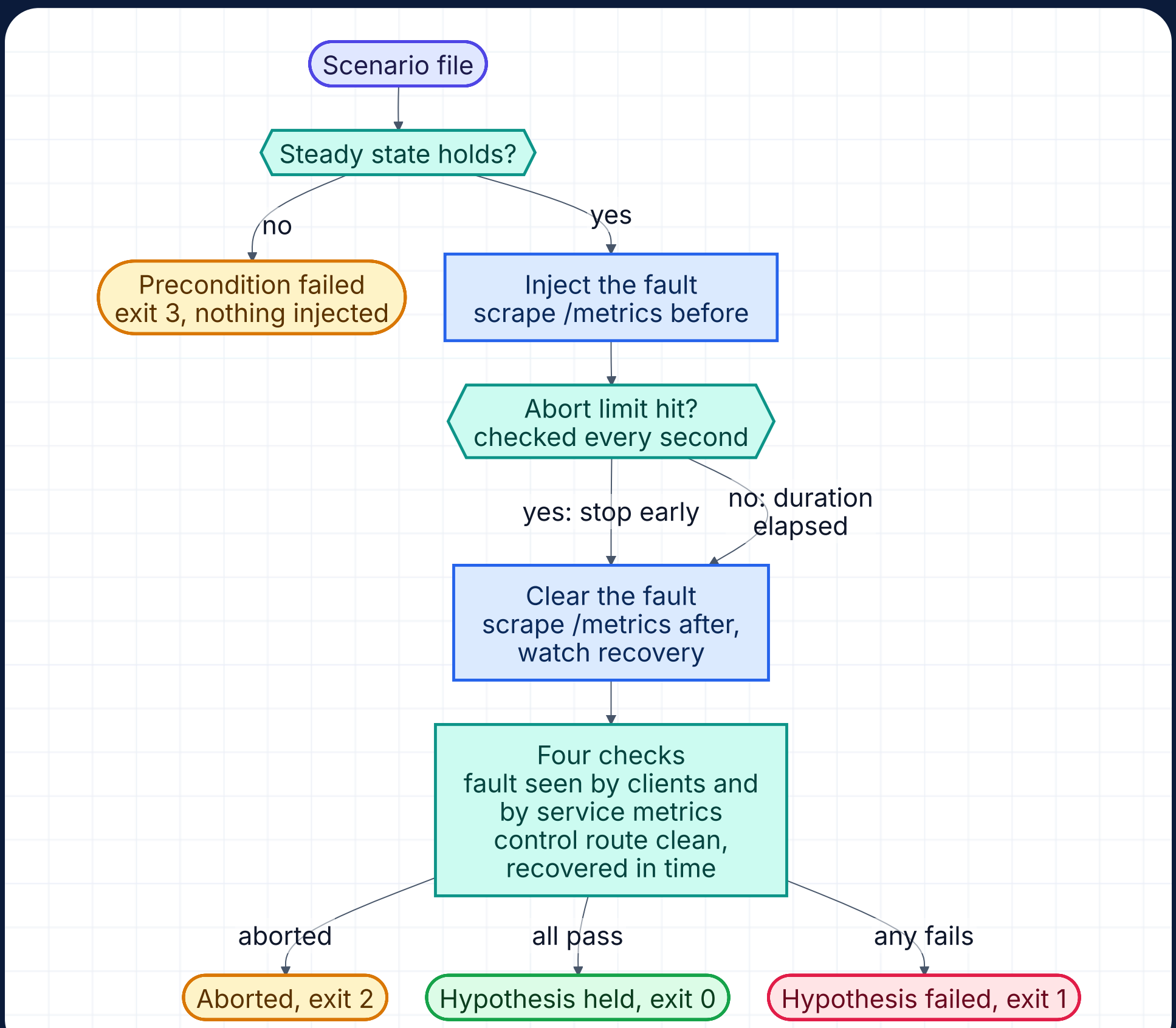
Pages wake someone. Tickets can wait.

Alertmanager routing, runbooks, postmortems and the action items that feed back into alerts.



A chaos experiment that checks its own claims

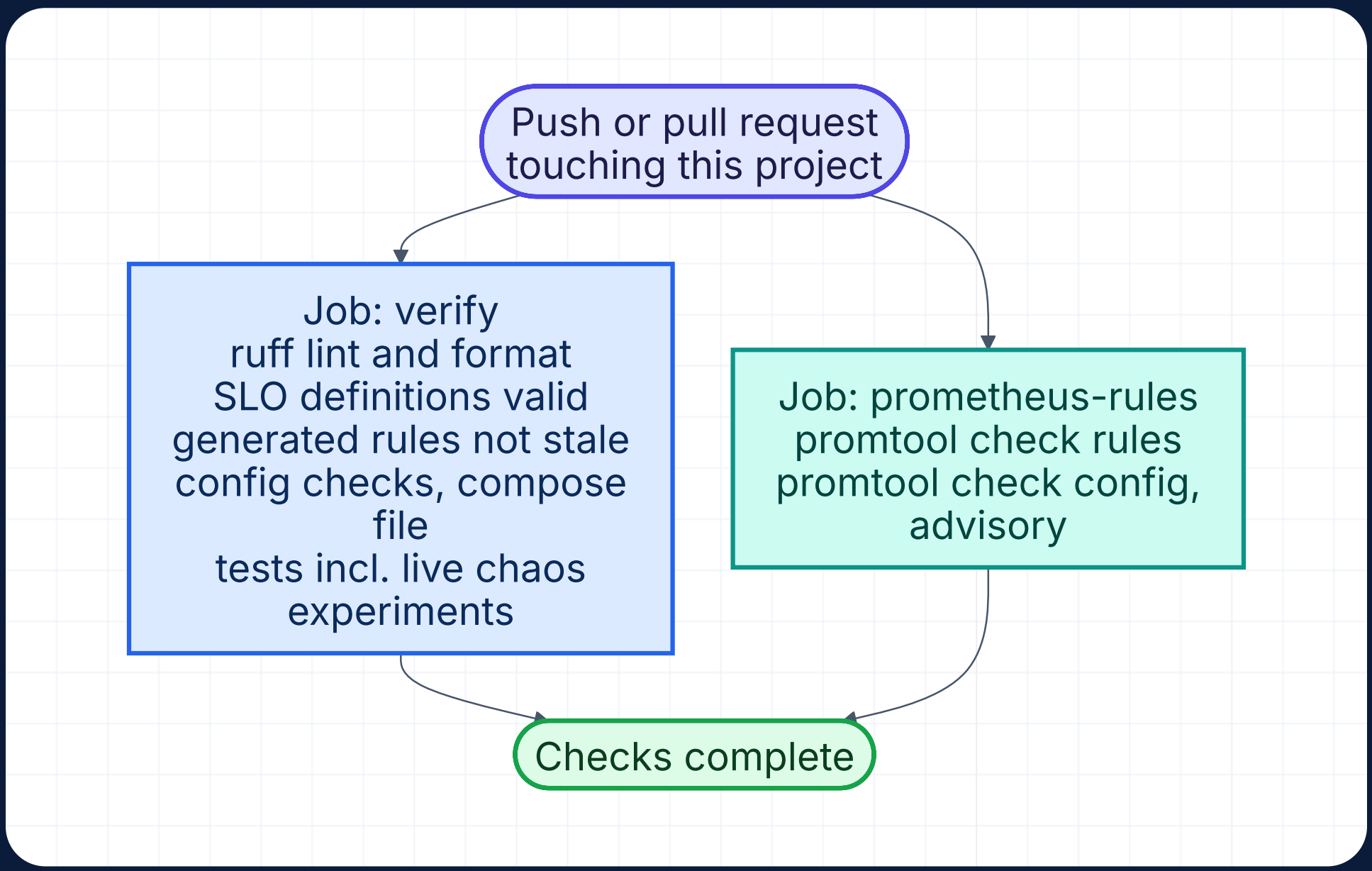
It refuses to start unless healthy, aborts on damage and checks that a control route stayed clean.



Scenario · Action · Check · Hypothesis held · Stopped early · Failed

Stale alert rules fail the build

CI regenerates the rules from the SLO file and runs Prometheus' own checker.



Trigger · Verify job · Prometheus job · Checks complete

What I ran, and what I did not

Run and passing

- 33 tests, including live chaos experiments
- Alert rules regenerate identically from the SLO file
- Two recorded game-day reports

Written, not yet run

- Prometheus evaluating the rules
- Alerts firing in Alertmanager
- Docker Compose stack

github.com/Joxejaime/projects-2026

projects/sre-reliability-lab, with tests, runbooks and these schematics